

**LAPORAN TUGAS BESAR**  
**MATA KULIAH TEORI BAHASA FORMAL DAN OTOMATA**  
**(IF2224)**



**TUGAS BESAR MILESTONE 1 : LEXICAL ANALYSIS**  
**KELOMPOK SHT-K02**

**Anggota :**

13524010 - Audric Yusuf Maynard Simatupang  
13524052 - Raynard Fausta  
13524062 - Nathan Edward Christoper Marpaung  
13524112 - Richard Samuel Simanullang

**SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA**  
**INSTITUT TEKNOLOGI BANDUNG**  
**2025**

## DAFTAR ISI

|                                             |           |
|---------------------------------------------|-----------|
| <b>LAPORAN TUGAS BESAR.....</b>             | <b>1</b>  |
| <b>DAFTAR ISI.....</b>                      | <b>2</b>  |
| <b>Teori Singkat.....</b>                   | <b>0</b>  |
| A. Lexical Analysis.....                    | 0         |
| B. Lexer.....                               | 0         |
| C. Token.....                               | 1         |
| D. Deterministic Finite Automata.....       | 1         |
| <b>Perancangan &amp; Implementasi.....</b>  | <b>2</b>  |
| Perancangan.....                            | 2         |
| A. Gambar Diagram Transisi Keseluruhan..... | 2         |
| Implementasi.....                           | 7         |
| <b>Pengujian.....</b>                       | <b>14</b> |
| Testing 1.....                              | 14        |
| Testing 2.....                              | 15        |
| Testing 3.....                              | 15        |
| Testing 4.....                              | 16        |
| Testing 5.....                              | 17        |
| Testing 6.....                              | 18        |
| Testing 7.....                              | 20        |
| Testing 8.....                              | 22        |
| <b>Kesimpulan dan Saran.....</b>            | <b>25</b> |
| Kesimpulan.....                             | 25        |
| Saran.....                                  | 25        |
| <b>Lampiran.....</b>                        | <b>26</b> |
| Link Repositori.....                        | 26        |
| Link Drawio.....                            | 26        |
| Pembagian Tugas.....                        | 26        |
| <b>Referensi.....</b>                       | <b>27</b> |

## **Teori Singkat**

### **A. Lexical Analysis**

Analisis leksikal (lexical analysis) merupakan tahap awal dalam proses kompilasi atau interpretasi program. Pada tahap ini, source code yang semula hanya berupa rangkaian karakter dibaca dari kiri ke kanan, lalu dipecah menjadi unit-unit bermakna yang disebut token. Tahap ini penting karena parser tidak lagi bekerja langsung pada karakter mentah, melainkan pada deretan token yang telah terstruktur. Selain itu, analisis leksikal juga umumnya mengabaikan karakter yang tidak berpengaruh terhadap struktur program, seperti spasi, tab, dan komentar.

Pada konteks compiler, hasil dari lexical analysis ini menjadi input untuk tahap syntax analysis. Analisis leksikal berfungsi sebagai penghubung antara teks mentah program dengan struktur sintaks yang akan diproses lebih lanjut.

### **B. Lexer**

Lexer atau lexical analyzer adalah komponen yang bertugas melakukan analisis leksikal. Lexer membaca karakter satu per satu dari file masukan, kemudian menentukan apakah rangkaian karakter tersebut membentuk token tertentu, misalnya keyword, identifier, konstanta bilangan, operator, atau delimiter. Setelah token dikenali, lexer mengeluarkan representasi token tersebut ke keluaran sehingga dapat diproses pada tahap selanjutnya.

Lexer pada milestone ini harus bekerja seperti Deterministic Finite Automata (DFA), sehingga pembacaan input dilakukan dengan satu karakter setiap saat. Artinya, program tidak boleh bergantung pada generator lexical analyzer seperti lex atau flex, melainkan harus mengimplementasikan logika pengenalan token secara mandiri di dalam program C/C++. Deteksi . Pendekatan ini membuat hubungan antara teori automata dan implementasi lexer menjadi terlihat secara langsung.

### **C. Token**

Token adalah unit terkecil yang memiliki arti dalam bahasa pemrograman. Setiap token biasanya direpresentasikan oleh jenis token dan nilai atau atribut token tersebut. Secara umum, kategori token yang dikenali lexer meliputi keyword, identifier, literal atau constant, operator, serta separator atau delimiter. Keyword adalah kata yang sudah memiliki makna khusus dalam bahasa, misalnya begin, end, atau if. Identifier digunakan untuk nama variabel, prosedur, atau entitas lain yang didefinisikan pengguna. Literal menyatakan nilai langsung seperti bilangan bulat atau riil, sedangkan operator dan delimiter membantu membentuk struktur ekspresi maupun pernyataan. Pengelompokan ini membuat parser dapat memahami peran setiap bagian program secara lebih sistematis.

### **D. Deterministic Finite Automata**

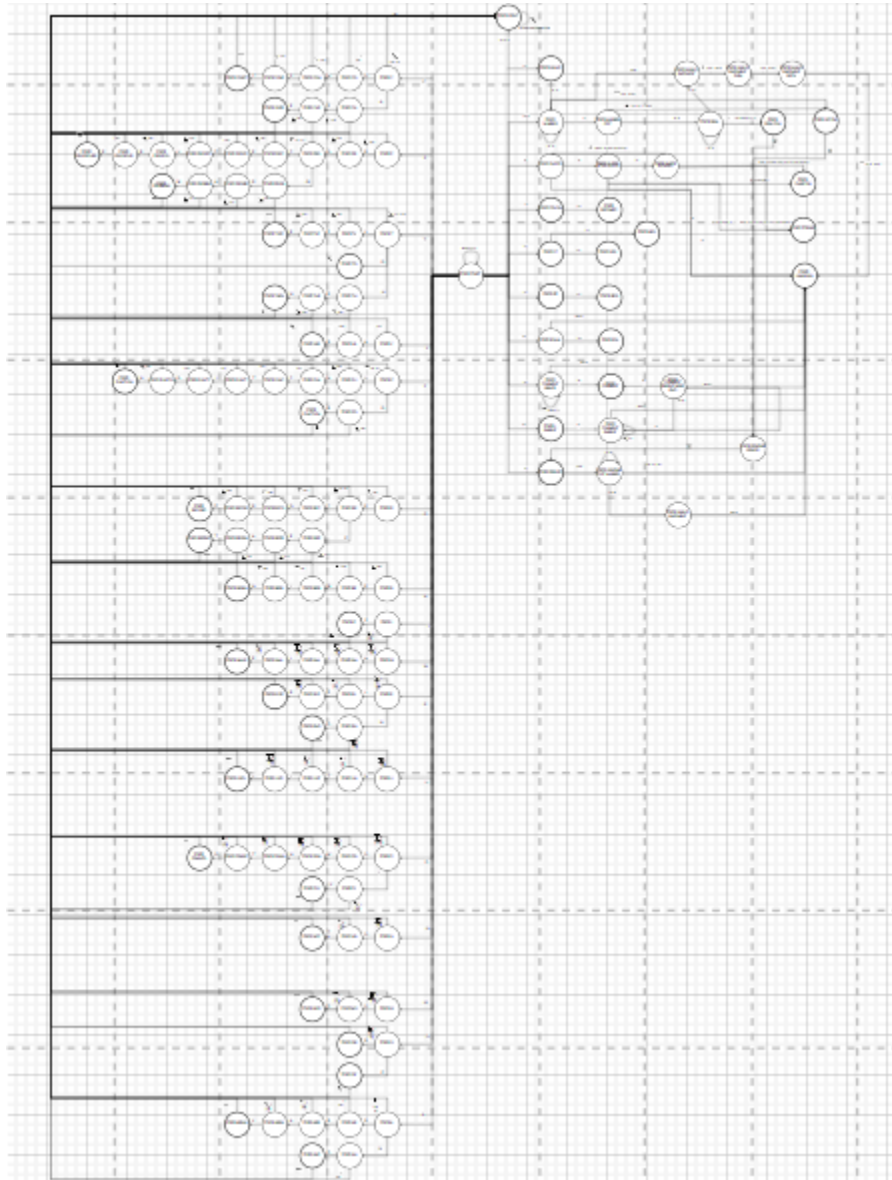
Deterministic Finite Automata (DFA) adalah model komputasi hingga yang terdiri atas himpunan state, alfabet input, state awal, transisi, dan state akhir. Disebut deterministik karena untuk setiap pasangan state dan simbol input, hanya ada satu transisi yang mungkin dilakukan. Sifat deterministik ini sangat sesuai untuk lexer, sebab lexer harus dapat memutuskan secara pasti langkah berikutnya hanya berdasarkan state saat ini dan karakter yang sedang dibaca.

Dalam implementasi lexer, setiap pola token dapat direpresentasikan sebagai jalur transisi pada DFA. Misalnya, ketika karakter pertama berupa huruf, automata berpindah ke state pembentuk identifier atau keyword; jika karakter pertama berupa digit, automata berpindah ke state pembentuk integer constant atau real constant; dan jika karakter berupa simbol khusus, automata berpindah ke state yang mengenali operator atau delimiter tertentu. Ketika automata mencapai state akhir, lexer dapat menyimpulkan bahwa sebuah token telah berhasil dikenali.

## Perancangan & Implementasi

### Perancangan

#### A. Gambar Diagram Transisi Keseluruhan



#### B. Penjelasan Rancangan DFA

Secara umum, alur DFA dimulai dari STATE\_START, yaitu keadaan awal saat program belum mengetahui token apa yang sedang dibaca. Dimulai dari state ini, program melihat karakter pertama dari masukan lalu menentukan jalur yang sesuai.

Apabila dari STATE\_START program menerima huruf pertama, seperti C, T, V, F, P, A, R, B, I, N, M, W, E, U, O, atau D, program membedakan pemrosesannya dari identifier biasa. Huruf-huruf tersebut masuk ke jalur khusus untuk memeriksa apakah rangkaian huruf tersebut membentuk keyword, seperti const, case, type, then, var, function, procedure, program, array, record, repeat, begin, if, not, while, end, else, until, of, or, div, do, atau downto. Jika urutan dan panjang hurufnya tepat, maka DFA berhenti di final state keyword yang bersesuaian. Namun, jika masih terdapat huruf atau digit atau susunan hurufnya menyimpang di tengah pemrosesan maka alurnya langsung dialihkan ke STATE\_IDENT sehingga token dianggap sebagai ident.

Apabila dari STATE\_START program menerima huruf lain selain yang di atas maka program langsung masuk ke STATE\_IDENT. Selama karakter berikutnya masih berupa huruf atau digit, DFA tetap berada di STATE\_IDENT. Ketika bertemu karakter yang bukan huruf atau digit, atau mencapai akhir proses maka DFA akan berhenti di STATE\_IDENT sebagai final state.

Apabila dari STATE\_START program menerima digit maka DFA beralih ke STATE\_NUMBER. DFA akan tetap berada di state itu selama membaca digit. Jika pembacaan berhenti karena bertemu karakter lain yang bukan (digit, titik, e, atau E) maka STATE\_INTCON akan menjadi final state dengan token yang dihasilkan adalah intcon. Namun jika setelah deretan digit muncul titik, DFA berpindah ke STATE\_NUMBER\_DOT. Pada state ini, apabila setelah titik muncul digit maka bilangan dianggap berlanjut menjadi bilangan riil dan DFA masuk ke STATE\_REAL. Selama DFA terus membaca digit maka DFA tetap berada di state tersebut lalu berhenti di FINAL\_REALCON saat bilangan selesai dibaca. Akan tetapi, pada kasus khusus jika setelah titik ternyata bukan digit maka DFA akan mengembalikan hasil sebelumnya sebagai intcon dan memasuki STATE\_PENDING\_PERIOD untuk memproses titik tersebut sebagai token period. Pada kasus khusus lainnya, kemunculan e atau E setelah angka tidak diperlakukan sebagai eksponen yang valid melainkan diarahkan ke rangkaian STATE\_INVALID\_EXPONENT yang pada akhirnya menghasilkan unknown. Apabila terdapat titik yang muncul sejak awal maka jika setelah . DFA membaca digit maka DFA masuk ke STATE\_LEADING\_DOT\_NUMBER.

Apabila dari STATE\_START menerima tanda petik tunggal ', DFA akan masuk ke STATE\_QUOTE. Pada state ini, setiap karakter biasa akan terus dikumpulkan dengan tetap berada di STATE\_QUOTE. Jika sebelum penutup muncul newline atau EOF, token langsung dianggap unknown karena kutip tidak tertutup dengan benar. Jika DFA menemukan tanda petik tunggal penutup, alur berpindah ke STATE\_QUOTE\_AFTER\_QUOTE. Pada state ini panjang isi yang telah terkumpul diperiksa. Apabila panjang isi tepat satu karakter maka hasil akhirnya adalah charcon dan apabila panjangnya tidak sama dengan satu, hasilnya adalah string. Terdapat juga jalur STATE\_QUOTE\_ESCAPED untuk menangani kasus adanya petik tunggal ganda di dalam isi.

Untuk operator dan tanda baca sederhana, DFA memprosesnya secara langsung. Dari STATE\_START, karakter +, \*, /, ), [, ], ,, dan ; langsung menuju final state masing-masing sehingga token seperti plus, times, rdiv, rparent, lbrack, rbrack, comma, dan semicolon dapat dikenali tanpa perlu state lanjutan. Tanda : masuk ke STATE\_COLON dan jika setelahnya ada = maka token yang dihasilkan adalah becomes, sedangkan jika tidak hasilnya hanya colon sebagai final state. Tanda < masuk ke STATE\_LT, jika diikuti = final statenya leq, jika diikuti >, hasilnya neq, dan jika tidak, hasilnya lss. Tanda > masuk ke STATE\_GT, jika diikuti = hasilnya geq dan jika tidak hasilnya gtr. Tanda = masuk ke STATE\_EQUAL dan jika diikuti = lagi maka hasilnya eql, tetapi jika tidak maka dianggap. Jika DFA awalnya membaca minus dan tidak ada digit setelahnya maka token minus dihasilkan dan jika setelahnya angka maka akan masuk ke STATE\_NUMBER.

Untuk komentar, DFA dapat memproses dua bentuk. Jika pada STATE\_START program membaca {, maka ia masuk ke STATE\_COMMENT\_BRACE dan akan terus berada di sana sampai menemukan }. Ketika } ditemukan, token comment dihasilkan, tetapi jika berkas habis terlebih dahulu dan } tidak terbaca maka hasilnya unknown. Bentuk kedua dimulai dari (. Dari STATE\_START, karakter ( akan berpindah ke STATE\_BRACKET. Jika setelahnya ada \*, maka itu dianggap awal komentar (\* ... \*) dan DFA masuk ke STATE\_COMMENT\_PAREN. Di dalamnya, setiap karakter biasa dibaca terus sampai muncul \* yang memindahkan proses ke

STATE\_COMMENT\_PAREN\_NEAREND. Jika muncul `)`, maka token comment dihasilkan. Jika tidak, DFA kembali lagi ke isi komentar.

## Implementasi

### A. Token

Masing-masing token disimpan di dalam sebuah ADT Token di file Token.c dan Token.h. ADT ini akan digunakan untuk mengenalkan token yang dibaca dan dikonversikan kepada file output. Tipe token akan disimpan di dalam ADT enum TokenType, yang kemudian akan diidentifikasi dan disimpan di dalam tipe data Token saat lexer memarsing file input. Fungsi tokenTypetoString diimplementasikan untuk mengenal tipe token yang dibaca dan dikonversikan menjadi bentuk string untuk dioutput ke file output.

```
typedef enum {  
    TOKEN_INTCON,  
    TOKEN_REALCON,  
    TOKEN_CHARCON,  
    TOKEN_STRING,  
    TOKEN_IDENT,  
    TOKEN_COMMENT,  
  
    TOKEN_PLUS,  
    TOKEN_MINUS,  
    TOKEN_TIMES,  
    TOKEN_IDIV,  
    TOKEN_RDIV,  
    TOKEN_IMOD,  
  
    TOKEN_ANDSY,  
    TOKEN_ORSY,  
    TOKEN_NOTSY,  
  
    ...  
    ...  
}  
TokenType;  
  
typedef struct {
```

```

    TokenType type;
    char lexeme[MAX_LEXEME];
} Token;

const char *tokenTypeToString(TokenType type) {
    switch (type) {
        case TOKEN_INTCON: return "intcon";
        case TOKEN_REALCON: return "realcon";
        case TOKEN_CHARCON: return "charcon";
        case TOKEN_STRING: return "string";
        case TOKEN_IDENT: return "ident";
        case TOKEN_COMMENT: return "comment";

        case TOKEN_PLUS: return "plus";
        case TOKEN_MINUS: return "minus";
        case TOKEN_TIMES: return "times";
        case TOKEN_IDIV: return "idiv";
        case TOKEN_RDIV: return "rdiv";
        case TOKEN_IMOD: return "imod";

        ...

        ...

        ...
    }
}

```

## B. MesinKarakter

Pembacaan dan parsing karakter dilakukan dengan menggunakan mesin karakter. Mesin karakter diimplementasikan di dalam file mesinkarakter.c dan mesinkarakter.h. Mesin ini akan menerima fileinput dan membaca karakter satu persatu untuk mengidentifikasi karakter dan token yang dibaca.

## C. Fileio

Fileio diimplementasikan untuk membantu dalam menerima file input agar dapat dibaca sekaligus membantu dalam menghandle output token yang diidentifikasi dan menulis ke

dalam sebuah file output. Fileio akan memeriksa file input dan memastikan file memiliki extension .txt, menerima isi file input agar dapat dihandle oleh mesinkarakter untuk dibaca, sekaligus membangun file output dari hasil pembacaan dan identifikasi token.

#### **D. Lexer**

Lexer berfungsi untuk menginisiasi pembacaan lewat mesinkarakter dan melakukan analisis leksikal untuk mendapatkan token-token dari input file. Lexer akan menginisiasi pembacaan dan menentukan token lewat pergantian STATE sesuai dengan DFA yang sudah digambar. Untuk membantu DFA berjalan, struct enum LexerState diimplementasikan yang berisi berbagai state dari DFA. Sebuah struct Lexer diimplementasikan udah menyimpan status mesin saat ready dan eof, menyimpan karakter yang sedang dibaca oleh mesin karakter, menyimpan jumlah pendingPeriod, serta menyimpan state DFA pada saat itu. Beberapa fungsi diimplementasikan untuk mempermudah analisis karakter dan token, seperti isDigitChar(), isIdentifierStart(), dan lainnya.

```
typedef enum {
    STATE_START,
    STATE_IDENT,
    STATE_NUMBER,
    STATE_REAL,
    STATE_QUOTE,
    STATE_COLON,
    STATE_LT,
    STATE_GT,
    STATE_EQUAL,
    ...
} LexerState;

typedef struct {
    bool ready;
    bool eof;
    char current;
```

```
    LexerState state;  
} Lexer;
```

Lexer akan menginisiasi dengan menetapkan status mesin menjadi ready dan menginisialkan state menjadi STATE\_START. Jika inisiasi gagal, maka mesin akan lompat menuju EOF dan menutup lexer dengan closeLexer(). Lexer yang berhasil inisiasi kemudian akan memanggil fungsi getToken() untuk melanjutkan pembacaan dan menganalisis token yang didapatkan.

Fungsi getToken() akan membaca karakter dan menentukan token yang sesuai dengan urutan pembacaan karakter. Setiap STATE dari mesin memiliki implementasi switch case masing-masing yang menentukan token yang sesuai dengan pembacaan karakter. Ketika mesin karakter menerima sebuah karakter, lexer akan memeriksa apakah karakter yang dibaca valid untuk lanjut ke state yang baru sesuai dengan DFA yang sudah dirancang dan mengganti state mesin sesuai dengan state yang valid dari pembacaan karakter tersebut. Jika mesin karakter menerima karakter yang tidak valid dan tidak memiliki alur jalan yang sesuai dengan rancangan DFA, maka state akan dipindahkan menuju STATE\_UNKNOWN, menset token Unknown dan memberhentikan lexer. Pembacaan karakter akan berlangsung hingga mendapatkan spasi atau escape character, dan kemudian akan mencatat token yang valid pada saat itu. Hal ini dilakukan berulang kali hingga lexer berhasil membaca seluruh input file dan mencatat token yang dihasilkan dari analisis leksikal. List token yang didapatkan kemudian akan digunakan dan di output ke dalam sebuah file.

### **E. Program Utama**

Program utama terletak pada main.c yang akan menghubungkan keseluruhan rancangan sistem, fungsi, dan ADT menjadi program utama yang akan menjalankan DFA. Program utama akan membaca file dengan nama yang diinput oleh user. Program utama akan memeriksa hingga nama file input valid, dan kemudian akan meminta nama file output yang diinginkan oleh user.

Program kemudian akan membangun file output berdasarkan path dan nama file yang sudah diinput, dan menginisiasikan lexer. Program utama kemudian akan memanggil fungsi tokenize() yang akan menerima hasil token lewat fungsi getToken() dari lexer dan menulis hasil token ke dalam file output lewat writeToken(). File output dan lexer kemudian ditutup, dan user dapat melihat hasil tokenisasi di dalam file output saat program berakhir.

```
static void writeToken(FILE *stream, const Token *tk) {
    if (tk->type==TOKEN_STRING) {
        fprintf(stream, "%s ('%s')\n", tokenTypeToString(tk->type),
tk->lexeme);
        return;
    }

    switch (tk->type) {
        case TOKEN_INTCON:
        case TOKEN_REALCON:
        case TOKEN_CHARCON:
        case TOKEN_IDENT:
        case TOKEN_COMMENT:
        case TOKEN_UNKNOWN:
            fprintf(stream, "%s (%s)\n", tokenTypeToString(tk->type),
tk->lexeme);
            return;
        default:
            fprintf(stream, "%s\n", tokenTypeToString(tk->type));
            return;
    }
}

static void tokenize(Lexer *lx, FILE *out) {
    while (true) {
        Token tk=getToken(lx);
```

```

        if (tk.type==TOKEN_EOF) {
            return;
        }

        writeToken(stdout, &tk);
        writeToken(out, &tk);
    }
}

int main(void) {
    while (true) {
        Lexer lx;
        FILE *out;
        bool validInput;
        char inputName[NAME_MAX_LEN];
        char outputName[NAME_MAX_LEN];
        char inputPath[IO_MAX_PATH];
        char outputPath[IO_MAX_PATH];

        do {
            validInput=false;
            printf("Masukkan nama file input (ketik 'exit' untuk
berhenti): ");
            (void)scanf("%255s", inputName);

            if (strcmp(inputName, "exit")==0) {
                printf("Program dihentikan.\n");
                return EXIT_SUCCESS;
            }

            buildMilestonePath(IO_INPUT_DIR, inputName, inputPath,
sizeof(inputPath));
            validInput=fileExists(inputPath);
            if (!validInput) {
                fprintf(stderr, "File input tidak ditemukan: %s\n\n",
inputPath);
            }
        }
    }
}

```

```

        } while (!validInput);

        printf("Masukkan nama file output: ");
        (void)scanf("%255s", outputName);

        (void)ensureMilestoneDirectories();
        buildMilestonePath(IO_OUTPUT_DIR, outputName, outputPath,
sizeof(outputPath));

        initLexer(&lx, inputPath);
        if (!lx.ready) {
            fprintf(stderr, "Gagal membuka file input: %s\n\n",
inputPath);
            continue;
        }

        out=fopen(outputPath, "w");
        if (out==NULL) {
            fprintf(stderr, "Gagal membuat file output: %s\n\n",
outputPath);
            closeLexer(&lx);
            continue;
        }

        printf("\nHasil tokenisasi:\n\n");
        tokenize(&lx, out);

        fclose(out);
        closeLexer(&lx);
    }
}

```

## Pengujian

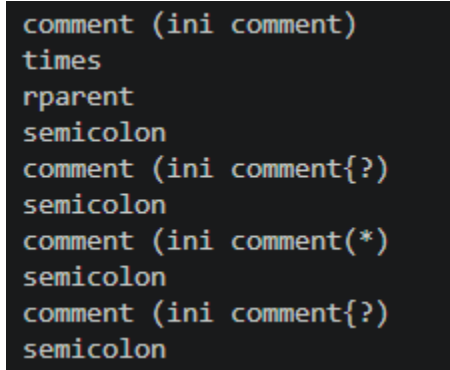
### Testing 1

Keterangan Input : Input bentuk program normal

| Input                                                                                                           | Output                                                                                                                                                                                                                                                                                                         |
|-----------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>program Hello;  var   a, b: integer;  begin   a := 5;   b := a + 10;   writeln('Result = ', b); end.</pre> | <pre>programsy ident (Hello) semicolon varsy ident (a) comma ident (b) colon ident (integer) semicolon beginsy ident (a) becomes intcon (5) semicolon ident (b) becomes ident (a) plus intcon (10) semicolon ident (writeln) lparent string ('Result = ') comma ident (b) rparent semicolon endsy period</pre> |
| Penjelasan : Output program sesuai dengan token yang ada pada tabel Arion                                       |                                                                                                                                                                                                                                                                                                                |

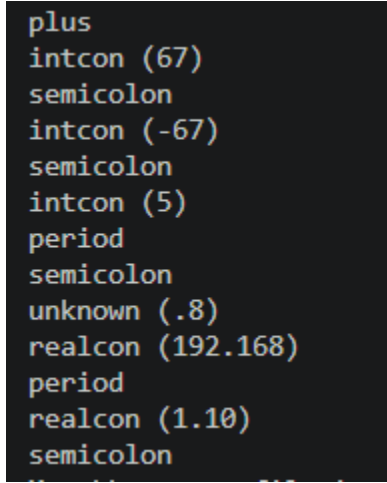
## Testing 2

Keterangan Input : Input comment dengan berbagai kondisi

| Input                                                                                                                                                                                                        | Output                                                                             |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| <pre>(*ini comment*) *);<br/>{ini comment{?};<br/>(*ini comment(**);<br/>(*ini comment{?*);</pre>                                                                                                            |  |
| <p>Penjelasan : comment berlaku untuk versi () dan {**}. Jika terdapat input karakter {} atau {**} dalam comment, maka commentnya berhenti ditengah sesuai urutan, sisa inputnya bukan termasuk comment.</p> |                                                                                    |

### Testing 3

Keterangan Input : Input integer dan real dengan berbagai kondisi

| Input                                                                                                                                                                                                                                                                                                                                                                                                                                                               | Output                                                                             |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| +67;<br>-67;<br>5. ;<br>.8<br>192.168.1.10;                                                                                                                                                                                                                                                                                                                                                                                                                         |  |
| <p>Penjelasan : Untuk +67, dianggap token plus, lalu intcon(67). Untuk kasus -67, dianggap* menjadi intcon(-67). Untuk kasus 5., maka akan dianggap intcon lalu period saja. Sementara itu pada kasus .8 maka dipilih unknown(.8). Untuk input dengan lebih dari 1 koma, maka yang dianggap realcon adalah angka yang punya angka di depan dan di belakang titik.</p> <p>Note* : untuk perubahan tambahan akan dilakukan di milestone berikutnya sesuai panduan</p> |                                                                                    |

## Testing 4

Keterangan Input : Input kasus Operator dan token keseluruhan

| Input                                                                                                                                                                                                                                                   | Output                                                                                                                       |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| a==2;<br>a==NOT b;<br>a==DIV c;                                                                                                                                                                                                                         | <pre>ident (a) eql intcon (2) semicolon ident (a) eql notsy ident (b) semicolon ident (a) eql idiv ident (c) semicolon</pre> |
| Penjelasan : Disini dipakai contoh operator == untuk menguji bagaimana proses antara final state ke final state. Setelah token tertentu dibaca, (dalam hal ini eql) maka akan ada reset sehingga bisa dibaca token selanjutnya (contohnya NOT dan DIV). |                                                                                                                              |

## Testing 5

Keterangan Input : Input program kesalahan leksikal

| Input                                                                                                                                                                                                                                                                                                                                                               | Output                                                                                                                                                                                                                                                             |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>var x;<br/>x = 10;<br/>y := 12e3;<br/>z := 'abc<br/>{ komentar tidak selesai</pre>                                                                                                                                                                                                                                                                             | <pre>varsy<br/>ident (x)<br/>semicolon<br/>ident (x)<br/>unknown (Invalid '=')<br/>intcon (10)<br/>semicolon<br/>ident (y)<br/>becomes<br/>unknown (12e3)<br/>semicolon<br/>ident (z)<br/>becomes<br/>unknown (Invalid string)<br/>unknown (Invalid comment)</pre> |
| <p>Penjelasan : Tanda = tunggal dianggap tidak valid dan menghasilkan unknown (Invalid '='). Bentuk 12e3 dianggap tidak valid dan menghasilkan unknown (12e3), lexer belum mendukung notasi eksponen. String 'abc yang tidak ditutup menghasilkan unknown (Unterminated string). Komentar { komentar tidak selesai menghasilkan unknown (Unterminated comment).</p> |                                                                                                                                                                                                                                                                    |

## Testing 6

Keterangan Input : Input menguji deklarasi dasar, identifier

| Input                                                                                                                                                                                                               | Output                                                                                                                                                                                                                                                                                                  |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>program Demo; const   angka := 123;   phi := 3.14;   huruf := 'j';   teks := 'IRK'; var   x: integer;   y: real;   c: char;   s: string; begin   x := angka;   y := phi;   c := huruf;   s := teks; end.</pre> | <pre>programsy ident (Demo) semicolon constsy ident (angka) becomes intcon (123) semicolon ident (phi) becomes realcon (3.14) semicolon ident (huruf) becomes charcon (j) semicolon ident (teks) becomes string ('IRK') semicolon varsy ident (x) colon ident (integer) semicolon ident (y) colon</pre> |

```
ident (real)
semicolon
ident (c)
colon
ident (char)
semicolon
ident (s)
colon
ident (string)
semicolon
beginsy
ident (x)
becomes
ident (angka)
semicolon
ident (y)
becomes
ident (phi)
semicolon
ident (c)
becomes
ident (huruf)
semicolon
ident (s)
becomes
ident (teks)
semicolon
endsy
period
```

Penjelasan : Program ini menguji pemakaian token-token programsy, constsy, ident, becomes, intcon, realcon, charcon, string, varsy, colon, semicolon, beginsy, endsy, period

## Testing 7

Keterangan Input : Input menguji operator logika, aritmatika, serta tanda kurung

| Input                                                                                                                                                                                                | Output                                                                                                                                                                                                                                                              |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>program Operasi; var   a, b, c: integer;   hasil: integer; begin   hasil := (a + b) * c - a div b MOD c;   if NOT (a == b) AND (b &gt; c) OR (a &lt; c) then     hasil := hasil / 2; end.</pre> | <pre>programsy ident (Operasi) semicolon varsy ident (a) comma ident (b) comma ident (c) colon ident (integer) semicolon ident (hasil) colon ident (integer) semicolon beginsy ident (hasil) becomes lparent ident (a) plus ident (b) rparent times ident (c)</pre> |

```

minus
ident (a)
idiv
ident (b)
imod
ident (c)
semicolon
ifsy
notsy
lparent
ident (a)
eql
ident (b)
rparent
andsy
lparent
ident (b)
gtr
ident (c)
rparent
orsy
lparent
ident (a)
lss
ident (c)
rparent
thensy
ident (hasil)
becomes
ident (hasil)
rdiv
intcon (2)
semicolon
endsy
period

```

Penjelasan : Program ini menguji pemakaian comma, colon, semicolon, beginsy, becomes, lparent, rparent, plus, times, minus, idiv, imod, ifsy, notsy, eql, andsy, gtr, orsy, lss, thensy, rdiv, intcon, endsy, period

## Testing 8

Keterangan Input : Input menguji Array, Record, Function, Procedure, of, do, to, downto

| Input                                                                                                                                                                                                                                                                                                        | Output                                                                                                                                                                                                                                                                                                                                                                                                                 |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> program StrukturData; type   Arr := array[1] of integer;   Rec := record     nilai: integer;   end; var   i: integer;   A: Arr; procedure Proses; begin   for i := 1 to 3 do     A[i] := i;   for i := 3 downto 1 do     A[i] := A[i] + 1; end; function Hitung; begin   A[1] := A[1] + 1; end. </pre> | <pre> programsy ident (StrukturData) semicolon typesy ident (Arr) becomes arraysy lbrack intcon (1) rbrack ofsy ident (integer) semicolon ident (Rec) becomes recordsy ident (nilai) colon ident (integer) semicolon endsy semicolon varsy ident (i) colon ident (integer) semicolon ident (A) colon ident (Arr) semicolon proceduresy ident (Proses) semicolon beginsy forsy ident (i) becomes intcon (1) tosy </pre> |

```
intcon (3)
dosy
ident (A)
lbrack
ident (i)
rbrack
becomes
ident (i)
semicolon
forsy
ident (i)
becomes
intcon (3)
downtosy
intcon (1)
dosy
ident (A)
lbrack
ident (i)
rbrack
becomes
ident (A)
lbrack
ident (i)
rbrack
plus
intcon (1)
semicolon
endsy
semicolon
functionsy
ident (Hitung)
semicolon
beginsy
ident (A)
lbrack
intcon (1)
rbrack
becomes
ident (A)
lbrack
intcon (1)
rbrack
plus
intcon (1)
semicolon
endsy
period
```

Penjelasan : Program ini menguji typesy, ident, becomes, arraysy, lbrack, intcon, rbrack, ofsy, recordsy, colon, semicolon, endsy, varsy, proceduresy, beginsy, forsy, tosy, dosy, downtosy, functionsy, plus, period

## **Kesimpulan dan Saran**

### **Kesimpulan**

Berdasarkan dari perancangan diagram, implementasi program, serta pengujian yang sudah dilakukan, disimpulkan bahwa program lexer yang dibangun berhasil memenuhi tujuan milestone 1 yaitu membuat lexical analyzer dengan pendekatan Deterministic Finite Automata (DFA). Program yang dibuat membaca karakter satu per satu lalu mengelompokkannya menjadi token-token yang sesuai pada file output.

Berdasarkan pengujian yang telah dilakukan, program mampu mengenali berbagai token penting seperti keyword, identifier, integer constant, real constant, string, character constant, comment, operator, dan delimiter. Selain itu, program juga mampu mendeteksi beberapa kesalahan leksikal, misalnya penggunaan = tunggal yang tidak valid, string yang tidak ditutup, komentar yang tidak ditutup, serta bentuk bilangan tertentu yang belum didukung. Hal ini menunjukkan bahwa lexer tidak hanya mampu melakukan tokenisasi terhadap input yang valid, tetapi juga memiliki mekanisme dasar untuk penanganan kesalahan leksikal.

Walaupun demikian, hasil pengujian juga menunjukkan bahwa masih terdapat beberapa keterbatasan pada implementasi saat ini. Beberapa bentuk input, seperti bilangan yang diawali titik seperti .8, atau penanganan beberapa kasus simbol tertentu belum sepenuhnya dapat didukung sesuai variasi bentuk leksikal yang mungkin muncul.

### **Saran**

Untuk milestone berikutnya, hasil tokenisasi yang dihasilkan lexer dipertahankan agar konsisten, namun menyesuaikan ke perubahan sesuai yang diberitahukan asisten. Hal ini akan berpengaruh sebagai input pada tahap selanjutnya. Pengujian juga dapat disarankan untuk menguji banyak kasus ekstrim dan kombinasi yang kompleks sehingga lexer yang dibangun dapat menjadi fondasi serta memudahkan pengerjaan pada milestone berikutnya.

## Lampiran

### Link Repositori

<https://github.com/RayapSunggal/SHT-Tubes-IF2224-2026>

### Link Drawio

[https://drive.google.com/file/d/15jLIYTM6gBD4iicKUvtt9UaPi\\_FYYjiE/view?usp=sharing](https://drive.google.com/file/d/15jLIYTM6gBD4iicKUvtt9UaPi_FYYjiE/view?usp=sharing)

### Pembagian Tugas

| Nama                              | Tugas                                        |
|-----------------------------------|----------------------------------------------|
| Audric Y. M.Simatupang - 13524010 | Merancang DFA bagian keyword                 |
| Raynard Fausta - 13524052         | Merancang DFA bagian karakter khusus         |
| Nathan E.C Marpaung - 13524062    | Merancang DFA bagian karakter khusus         |
| Richard S Simanullang - 13524112  | Merancang DFA bagian number dan kasus khusus |

## Referensi

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. 2007. *Compilers: Principles, Techniques, and Tools*. 2nd ed. Boston: Pearson Addison-Wesley.

Hopcroft, J. E., Motwani, R., & Ullman, J. D. 2006. *Introduction to Automata Theory, Languages, and Computation*. 3rd ed. Upper Saddle River, NJ: Pearson Prentice Hall.

Stanford University. n.d. *CS143 Handout: Lexical Analysis*. Stanford CS143

TutorialsPoint. n.d. *Compiler Design - Lexical Analysis*. Tersedia pada:

[https://www.tutorialspoint.com/compiler\\_design/compiler\\_design\\_lexical\\_analysis.htm](https://www.tutorialspoint.com/compiler_design/compiler_design_lexical_analysis.htm)